Handover Document

University of BRISTOL

Estimating Carbon

April 30 2026

Project Overview

There are already tools and plugins that measure the carbon footprint of code builds, infrastructure usage, or web performance. Some VS Code extensions even estimate the environmental cost of local development. However, none of them track AI token usage in context, and certainly not in a way that separates tokens sent to LLMs during app runtime inside the IDE (for example, while running and testing code) and tokens sent to LLMs for development support (the behind-the-scenes calls that power Copilot chat and AI-assisted features).

This is the gap our solution addresses. By focusing on these two AI-specific metrics, we can give developers and teams visibility over both the carbon and cost of their AI usage, making it possible to make smarter, lower-impact choices earlier.

Features

Carbon emissions are tracked in real time as they are generated from the use of AI in an Integrated Development Environment (IDE).

The results from this tracking are immediately displayed in the status bar (fig 1) of vsCode, showing warning colours if this call is of a particularly high level. There is also a comprehensive dated log of all emissions (fig 2), the associated model, and the date of the AI call in the side bar.

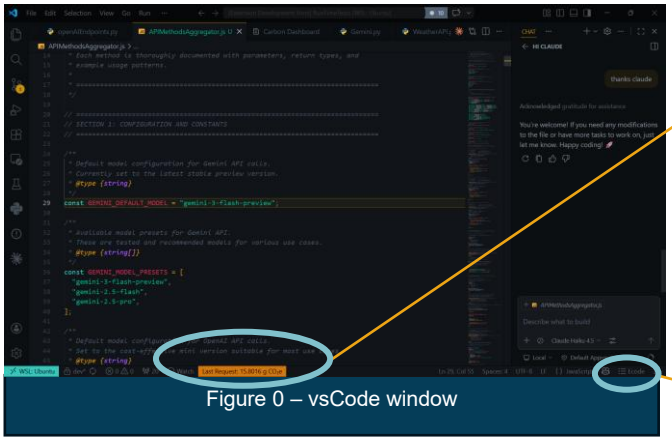

Figure 0 – vsCode window

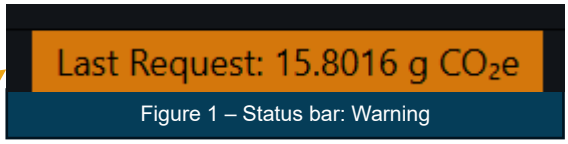

Figure 1 – Status bar: Warning

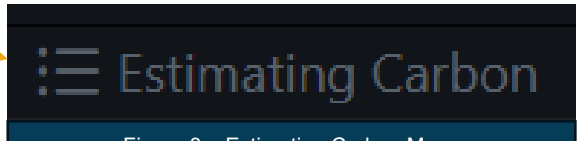

Figure 3 – Estimating Carbon Menu

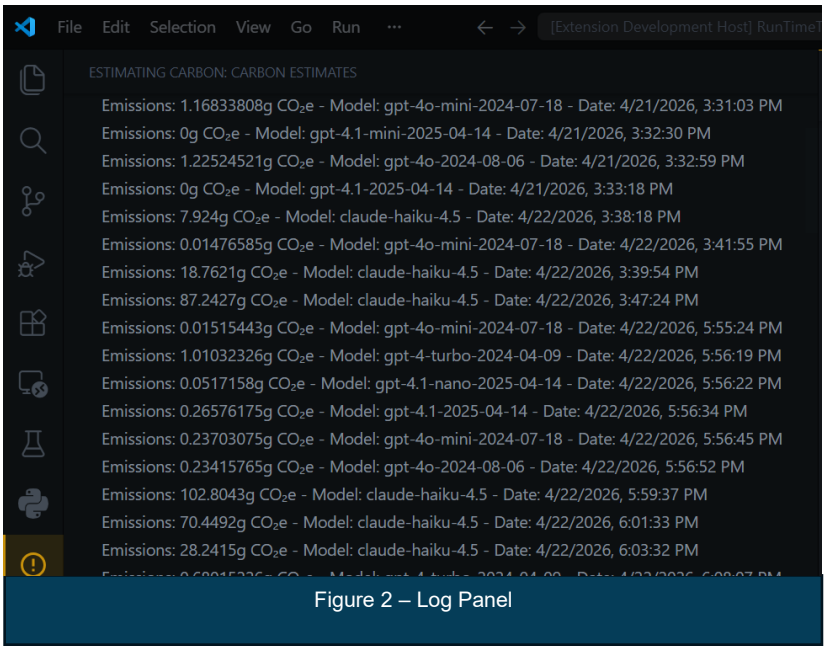

Figure 2 – Log Panel

These results are tracked during development time (when a developer is coding). Any* usage of Github Copilot will trigger the extension. Token count will be analysed and an emission calculated.

The emissions are also tracked during run time (when a developer is running a file). A custom runtime terminal is accessible through the Estimating Carbon Menu (fig 3). Running a file in this terminal will let the extension be able to track all AI that is called within the running file.

A dashboard that shows information on the emissions produced and models used is openable through the menu and updates in real time. The data is viewable by branch or by time as necessary.

Within this dashboard are many elements including

- A radar chart to display differences of model usage by branch (fig 4)
- A pie chart to display the most used models across a project (fig 5)
- A budget indicator that allows users to set their own “carbon budget”. The progress bar turns amber, then red as you approach your limit (fig 6)
- A dynamic bar chart showing emissions by time and call (fig 7)
- A cumulative timeline graph showing the progression of emissions on each branch versus all others
- A heatmap to display the previous emission history with up to a year of previous data (fig 8)
- 3 contextualising aids that let users see what their emissions mean in the real world (fig 9)
- An average request cost indicator (fig 10)

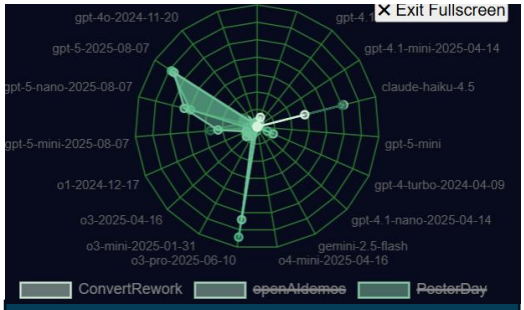

Figure 4 – Radar Chart

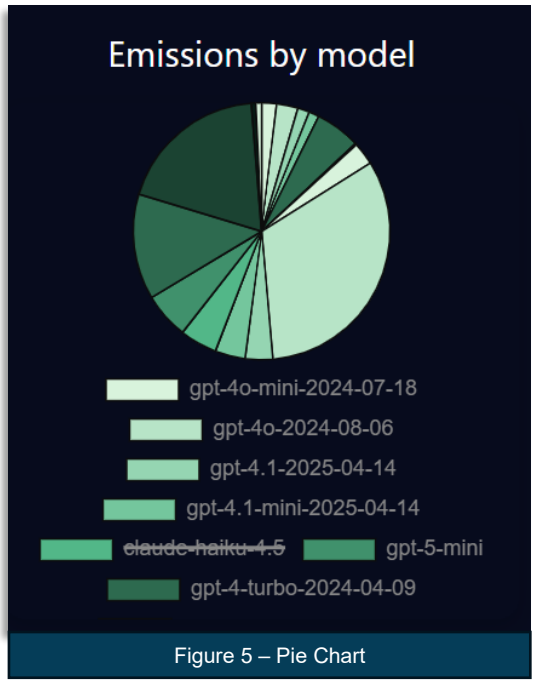

Figure 5 – Pie Chart

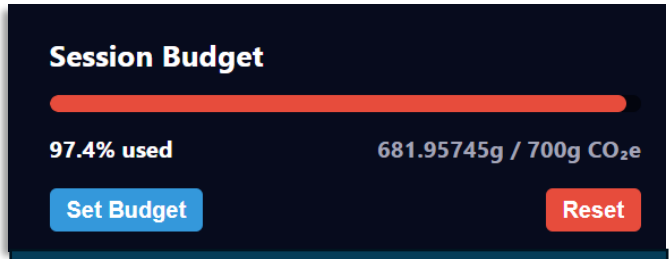

Figure 6 – Budget Bar

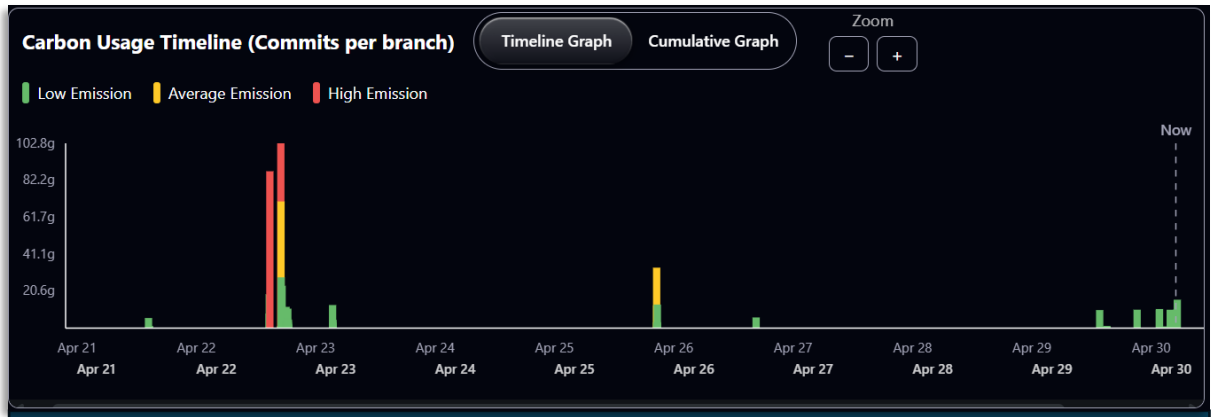

Figure 7 – Timeline Graph

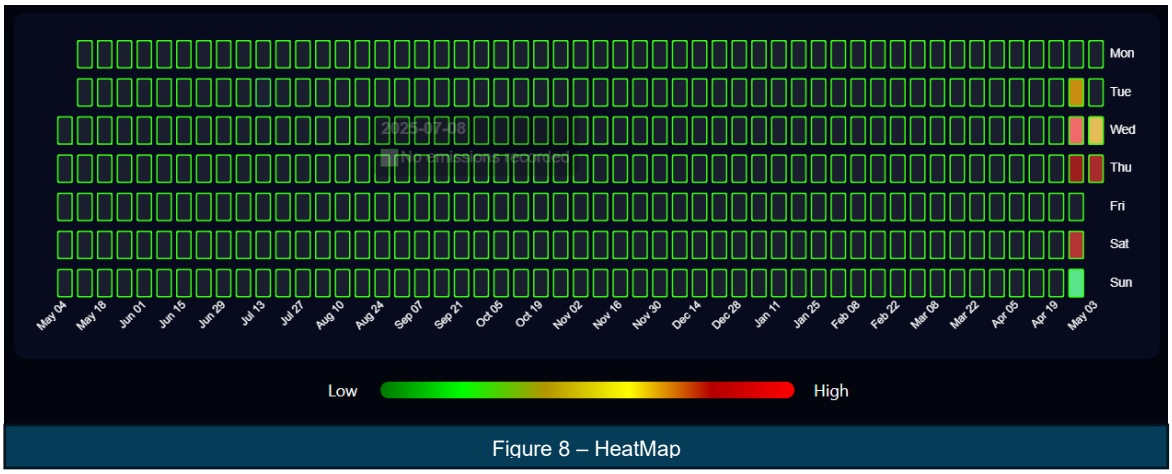

Figure 8 – HeatMap

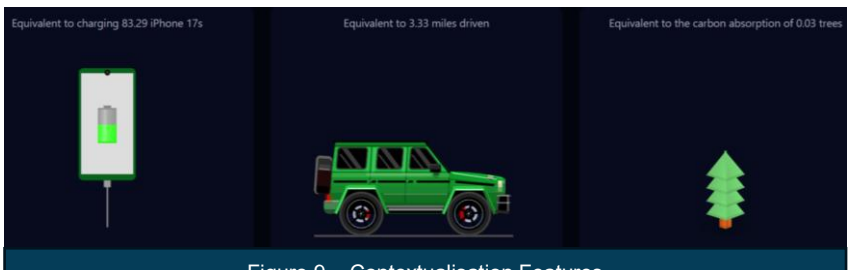

Figure 9 – Contextualisation Features

Figure 10 – Average Request Indicator

USAGE INSTRUCTIONS

Run Time

To start runtime analysis, first run the “Start Proxy Interceptor” command, which starts the proxy used to catch outgoing calls made to LLMs. Next, run the “Open Runtime Terminal” command. All calls must be made through this terminal with the correct environment variables to be tracked by the extension.

Development Time

For development time, first run the “Developer: Set Log Level” command, followed by “GitHub Copilot Chat” and then “Trace”. The extension will prompt the user to do this when it is first opened. Calls made through Copilot are then tracked and stored when a file is saved, or the “Refresh carbon data” command is run.

UI

- To access the dashboard, run the “Open Carbon Dashboard” command either through the command palette or the dedicated Ecode menu on the bottom left. The graphs and other data are updated when either a call is stored from runtime or when the refresh command is run for devtime.
- To view historical carbon data over time, select “Timeline Graph” from the top graph window. This will display metadata about individual commits over time.
- To view a cumulative total of carbon emissions on each branch, select “Cumulative Graph” from the top graph window. This will display a line graph showing the cumulative total emissions for each branch as a line graph.
- To view emissions by model, hover over the pie chart segment for each model type to view the total carbon emissions for that model.
- To set a carbon budget for the current session, select “Set Budget” from the budget widget and enter the desired amount. The display will change colour depending on how much of the budget has been used – 33% or below shows as green, 33%-90% shows as yellow and above 90% shows as red.
- To view total emissions on a given day, hover over the chosen day on the heat map, which show the date and total emissions for that day. In dark mode, the colours on the heatmap get paler as emissions increase, and vice versa for light mode. Emissions below 1% of the session budget show as green, between 1%-5% as yellow and above 5% as red.

ISSUES

Images don't produce a carbon cost - image json response doesn't give tokens, since pricing is done by image size, quality, model used for generation, and quality. These fields have all been parsed, but the research and implementation for carbon measuring is in progress.

In some cases, the zoom functionality on the timeline graph doesn't present the user with a scrollable interface. This results in the lack of granular analysis of data outside the view window.

Run time analysis does not currently work for Windows machines due to security conflicts.

Development time analysis is currently only compatible with GitHub Copilot.

There is a known issue with conversion ratios. The data we currently have given emission-per-token data that can get smaller over time with models with more/less reasoning or context, but we do not have enough data points to measure how it scales accurately.

FUTURE IMPLEMENTATIONS

Although text is tokenised for Gemini and older ChatGPT (< GPT 5) integration, caching prevents the capture of all interactions - for e.g. function calls. At our current stage, we have functionalities that successfully identify these older GPT and Gemini models and tokenise the produced output with the correct model's algorithm. This functionality currently does not visibly affect our extension since the data would imply that old GPT and Gemini models are more eco-friendly than they are. Future work could build on this current methodology to capture all tokens through a different method and return them to the dashboard.

Unlike text processing, where estimating energy consumption is straightforward, image inputs do not follow a token-based structure, instead returning fields such as size, quality, and number of images produced. This makes their energy usage harder to quantify and does not lend itself to a token-based approach. While there is developing support to track image metrics, it was not integrated within the scope of this project. Within serverWorker.ts there are commented out lines that pass the flow of logic towards an (un-implemented) conditional statement. Future work could focus on developing a method to process and convert image metadata into emissions.

Water usage data is currently missing from the registry limiting the analysis of environmental impact. However, the system has been integrated with a commented interface to support this data and can be enabled once all inputs are provided.

Input and output tokens are currently being treated separately. While our current system supports using appropriate methodologies and getting the total, this functionality remains unimplemented and is a nice future enhancement.

The cumulative graph can be enhanced by adding features like annotations for key project milestones such as major merges or releases, and/or adding another mode of display – e.g. a bar graph – to contrast the cumulative graph with a non-cumulative

graph to better analyse activity. It can also be enhanced by adding the hover feature, just like in the timeline graph.

MAINTENANCE

```
├── Images # architecture diagrams
├── ProjectNotes # meeting minutes and other project notes
├── README.md # main repo readme
├── Methodology.md # documentation for carbon calculations
└── vsCodeExt # main directory for VS Code extension
    ├── package.json # config file containing extension metadata
    ├── models.json # emissions conversion ratios for different models
    ├── LICENSE.md
    ├── src
    │   ├── budget.ts # interface for Call structure and usage calculations
    │   ├── convert.ts # handles conversion of token data to carbon emissions
    │   ├── dashboard.ts # main UI file
    │   │   ├── extension.ts # main UI features and commands
    │   │   ├── logCapture.ts # handles collection and parsing of Copilot log files
    │   │   ├── proxyServer.ts # starts server, receives parsed information
    │   │   ├── serverWorker.ts # listens on proxyServer and handles parsing
    │   │   ├── state.ts # flag for runtime testing
    ├── webview
    │   ├── dashboard.js # functionality for dashboard
    │   ├── style.css # styling for dashbord
    │   ├── graph.js # logic for timeline graph
    │   ├── darkmode.js # logic for darkmode
    └── tsconfig.json
```

Path	Contents
vsCodeExt/src	Tests The backend code including all supported token collection and conversion to emission
vsCodeExt/src/dashboard.ts	The HTML for the dashboard as well as the link between backend and frontend.
vsCodeExt/webview	The CSS and JavaScript for the dashboard

MEMENTOS

The budget data is stored using VS Code's memento API, which provides persistent storage without requiring an external database.

CALL INTERFACE

Calls made are stored in the format of a call interface which includes a model, time stamp, emissions, and a branch.

```
export interface Call {
    Model: string;
    DateTime: number;
    Emissions: number;
    Branch?: string;
}
```

All models and model conversion rates are stored in models.json along with two limits including how they grow differently depending on the input. This can be edited by the user if more accurate data becomes available in the future.

NPM SCRIPTS

Our package.json contains local scripts that allow for various functionality:

Npm install – installs all relevant dependencies

Npm run compile – compiles the extension

Npm run test – runs the test files and produces a code coverage report. Current test coverage is 88.64% (src)

Npm run release – packages the extension into an executable .vsix file, versioned according to the package.json version number

> Note: Semantic release is used to auto-increment the package version. The relevant documentation for this usage is laid out below:

“Commit message format

semantic-release uses the commit messages to determine the consumer impact of changes in the codebase. Following formalized conventions for commit messages, semantic-release automatically determines the next semantic version number, generates a changelog and publishes the release.

By default, semantic-release uses Angular Commit Message Conventions. The commit message format can be changed with the preset or config options of the @semantic-release/commit-analyzer and @semantic-release/release-notes-generator plugins.

Tools such as commitizen or commitlint can be used to help contributors and enforce valid commit messages.

The table below shows which commit message gets you which release type when semantic-release runs (using the default configuration):

Commit message	*Release type*
fix(pencil): stop graphite breaking when too much pressure applied	*~~Patch~~ Fix Release*
feat(pencil): add 'graphiteWidth' option	*~~Minor~~ Feature Release*
perf(pencil): remove graphiteWidth option *BREAKING CHANGE: The graphiteWidth option has been removed.* *The default graphite width of 10mm is always used for performance reasons.*	*~~Major~~ Breaking Release* *(Note that the BREAKING CHANGE: token must be in the footer of the commit)*

Contact us:

For any questions regarding the extension or future development please contact:

- Max Davies (cg24012@bristol.ac.uk)

- Jacob Connor (gn24034@bristol.ac.uk)
- Morgan Parry (vi24348@bristol.ac.uk)
- Iman Hadi (jp24368@bristol.ac.uk)
- Aayush Bhalerao (in24486@bristol.ac.uk)
- Hao Ni (wx24939@bristol.ac.uk)